RFC — Federated LLMOps Platform

---

1. Overview

This RFC proposes a **Federated LLMOps Platform** architecture designed to support:

- Retrieval-Augmented Generation (RAG)
- Prompt versioning and management
- Evaluation pipelines (LLM evaluation)
- Observability and tracing (Langfuse)

The system is implemented as a **modular, containerized platform** running locally via Docker, with a strong emphasis on **extensibility, reproducibility, and LLMOps best practices**.

---

2. Goals

Primary Goals

- Provide a production-like LLM system locally
- Enable prompt versioning and traceability
- Support RAG workflows using vector search
- Implement evaluation pipelines for prompt/model comparison
- Ensure observability of all LLM interactions

Non-Goals

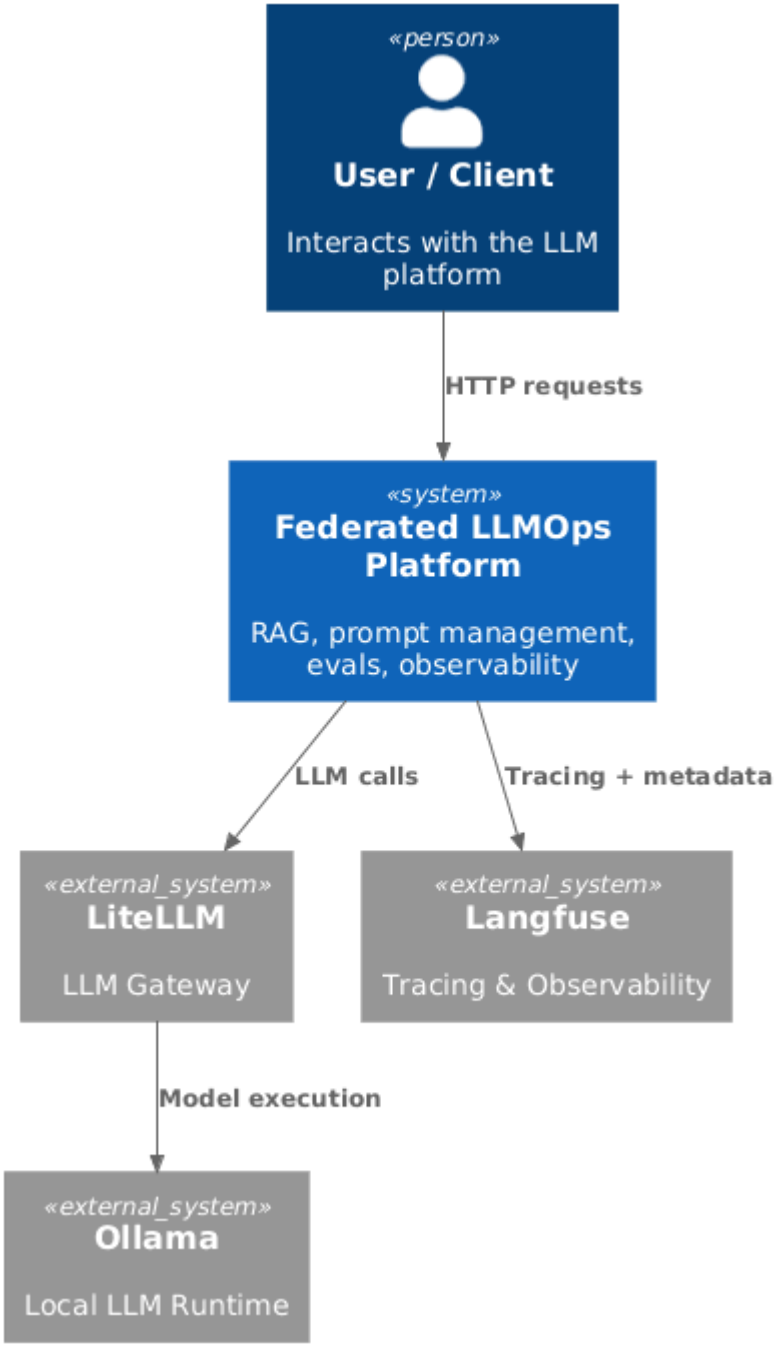
- LLM training or fine-tuning
- Distributed/cloud-native deployment (future work)

- High-scale serving optimization

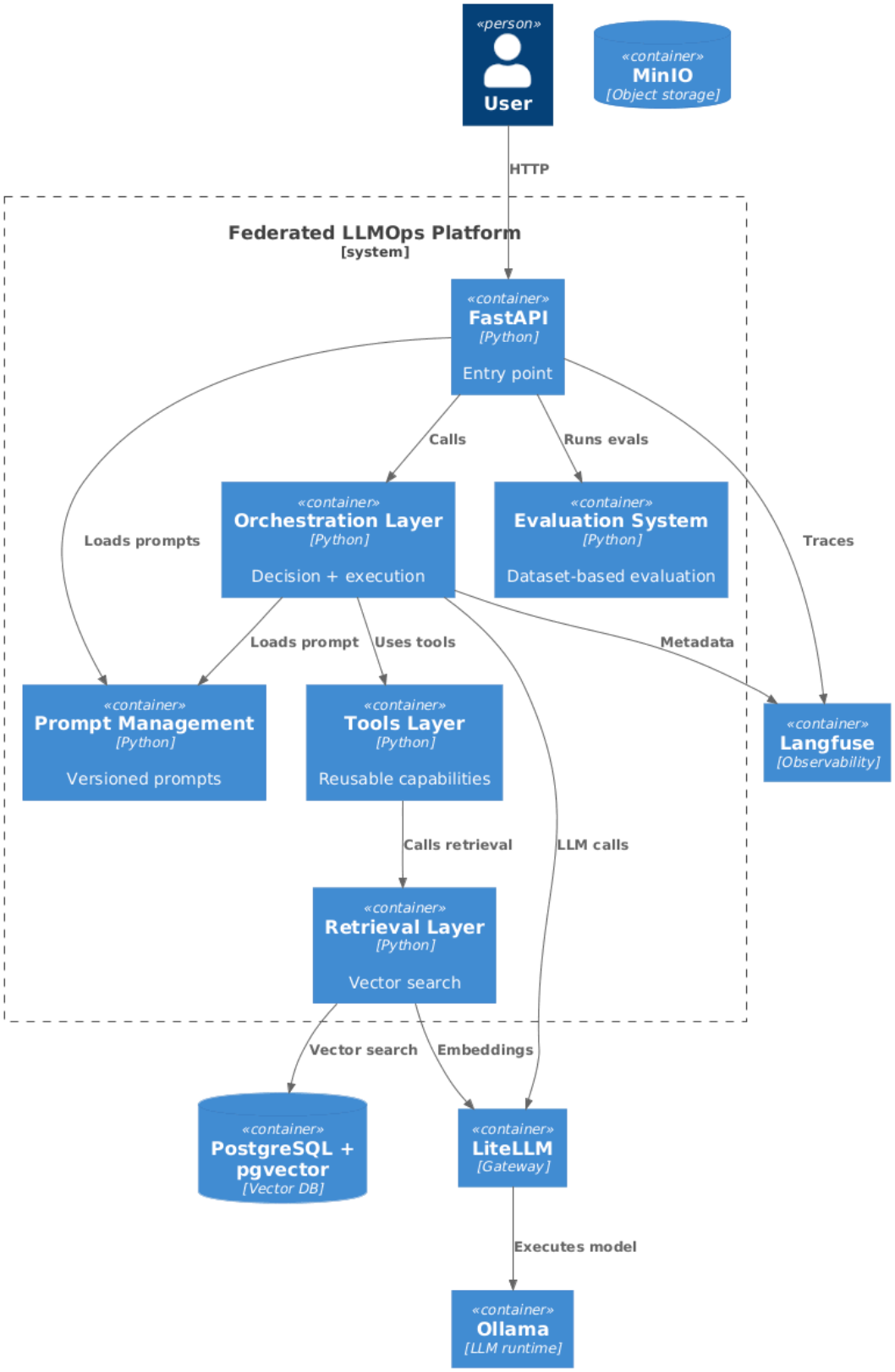
---

3. System Architecture

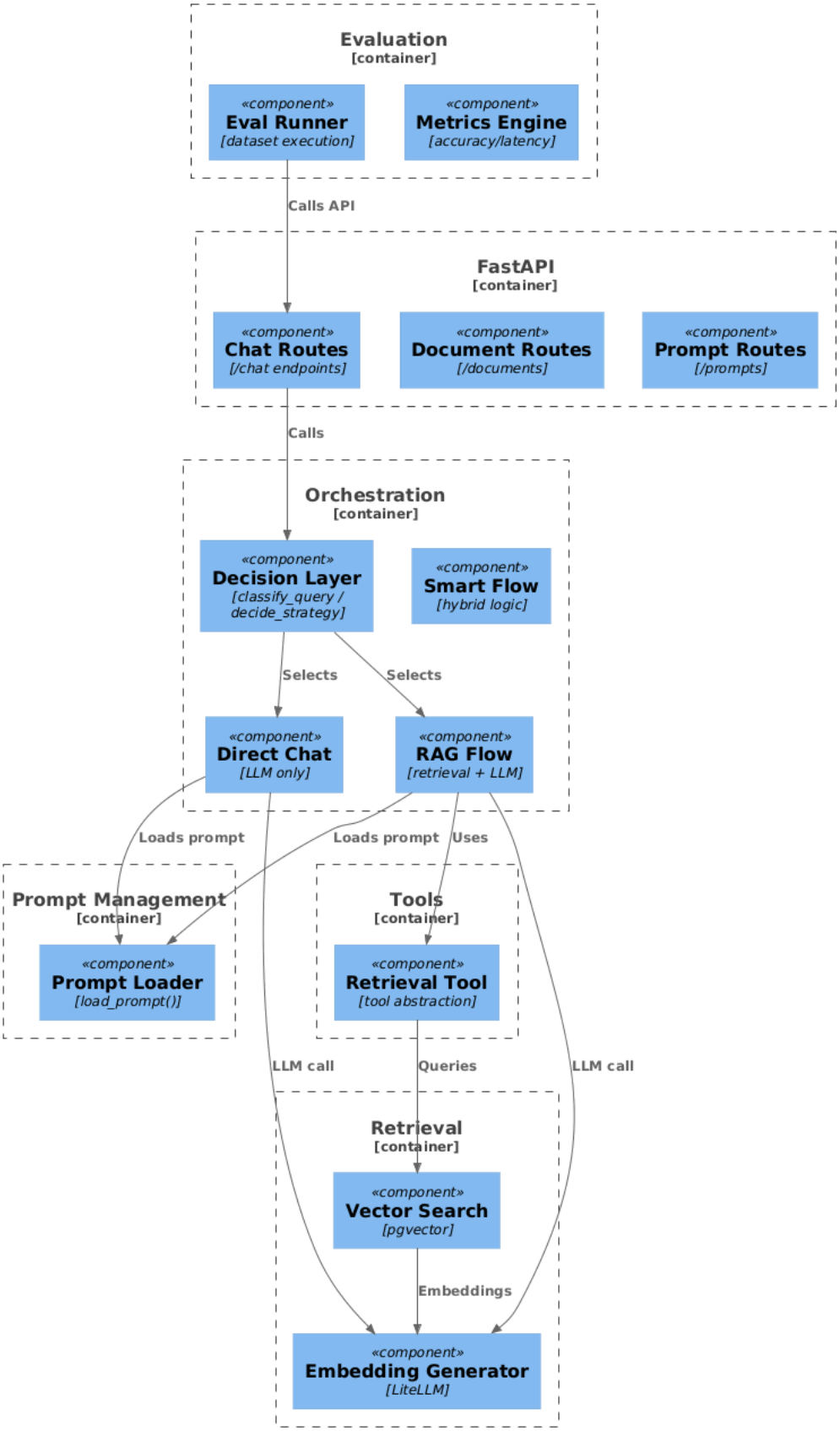
3.1 C4 Level 1 — System Context



3.2 C4 Level 2 — Container Diagram



3.3 C4 Level 3 — Component Diagram



---

4. Core Flows

4.1 RAG Flow

1. Request → `/chat/rag`
2. Orchestration decides strategy
3. Prompt loaded (versioned)
4. Retrieval:
    - embedding via LiteLLM
    - similarity search via pgvector
5. Context injected
6. LLM call executed
7. Response returned
8. Trace logged

4.2 Document Ingestion

1. Upload document
2. Store in MinIO
3. Chunk document
4. Generate embeddings
5. Store in Postgres (pgvector)

4.3 Evaluation Flow

1. Load dataset
2. Execute full pipeline per item
3. Compute:
    - correctness
    - latency
    - similarity
4. Store results
5. Compare prompt versions

---

5. Prompt Management

Features

- versioning (`v1`, `v2`)
- metadata-driven execution
- dynamic loading (`load_prompt`)
- Langfuse trace integration

---

6. Evaluation System

Capabilities

- dataset-based eval
- real pipeline execution
- prompt comparison

- CLI interface

Limitations

- no semantic judge (future)
- no automated gating

---

7. Architectural Decision — Federated LLMOps

7.1 Definition

A **federated architecture** separates concerns into modular domains:

- prompts
- orchestration
- tools
- retrieval
- evals

Each evolves independently.

7.2 Why Federated?

Modularity

- swap retrieval backend
- change prompts without touching code
- add tools independently

Extensibility

Supports:

- agents
- multi-model routing
- advanced evals

Alignment with modern patterns

- similar to **Data Mesh (Fowler)**
- aligns with **LLMOps modular pipelines (AWS, Microsoft)**

---

8. Comparison with Other Architectures

Monolithic

API → LLM

❌ No observability
❌ No modularity

Pipeline-based

API → Retrieval → LLM

❌ Rigid
❌ Hard to extend

Microservices-heavy

API → multiple services

❌ Over-engineered
❌ High latency

Federated (this)

API → orchestration → modules

✅ Balanced
✅ Extensible
✅ Simple deployment

---

9. Trade-offs

Decision	Trade-off
In-process orchestration	less scalable, simpler
Local models	no cloud scale
Simple evals	less accuracy
Docker Compose	not production-scale infra

---

10. Future Work

- Model routing (cost-aware)
- Advanced evals (LLM-as-judge)
- Memory layer
- Async pipelines
- Guardrails / safety

---

11. References

- AWS — Architect a mature generative AI foundation
- Microsoft — LLMOps
- Martin Fowler — Data Mesh